

GB reweighting of MC to data on reconstructed direction

Documentation of the pipeline and the first ROC sanity check

Holger

Purpose

This note documents the pipeline that produces the plot `roc_mc_vs_data_before_after.png` and the two weight files `GB_and_base_weights_stopped.csv` and `GB_and_base_weights_through.csv`. The goal is to reweight the Monte Carlo (MC) muon sample so that its reconstructed (*zenith*, *azimuth*) distribution matches that of data, separately for events classified as stopped and through-going. All other reconstructed observables (energy, *z*-position, track length, charge, N_{hits} , ...) are deliberately kept as *unreweighted* variables, so that they can later be used as blind validation tests of how well MC reproduces data.

1 Inputs

MC sample. File:

`MC_vs_BS_analysis/MC/muons_1305k_130000_720k_139008.db`

Contains ~ 2.0 M simulated muon events with standard pulse maps (`SplitInIcePulses`) and a truth table with the physics weight `norm_class_this_db_osc_weight` (oscillation/flux weighting, used as the MC *base weight* throughout).

Data sample. File:

`MC_vs_BS_analysis/Data/data_IC86.21_withrates.db`

Real IceCube data from IC86 with ~ 4 M events and the per-event rate weight `subrun_weight` (used as the data *base weight*). The data sample is pre-filtered by a muon PID score: we retain only events with

$$\text{pid_muon_logit_data} = \text{logit}(\text{pid_muon_pred}) > 5,$$

where `pid_muon_pred` is stored in

`data_IC86_2021_with_subrun_PID_888826.csv`

and `logit` is computed with a small ε -stabilisation to avoid $\log(0)$. This leaves ~ 1.9 M high-purity data muons.

2 Reconstructions applied

Two transformer models were run in inference mode on *both* the MC and the data sample, with identical pipelines so that MC and data are compared apples to apples.

Stopped / through-going classifier. Model:

ThroughOrStopped_muon/results/stopped_transformer_2M/best_model.pt

This is a pulse-level transformer that outputs a single logit per event. We apply a σ and keep the score. Outputs are written to

ThroughOrStopped_muon/inference/output/stopped_recon_*.csv

with columns event_no, stopped_logit, stopped_score, stopped_pred.

Zenith / azimuth regressor. Model: Inar's direction transformer,

Classifiers/Inars_zenith_azimuth_transformer_recon/results/transformer_720k_muons/best_model.pt

The training configuration uses position-mode pulse grouping on the feature set [dom_x, dom_y, dom_z, dom_time, charge, width], so the inference script forces the *same* feature set for both MC and data (instead of letting the auto-detect pick sensor_id mode on the data DB, which would have used a different collator). Outputs are written to

MC_vs_BS_analysis/zenith_azimuth_inference/output/zenaz_recon_*.csv

with columns event_no, zenith_pred, azimuth_pred.

3 GB reweighting procedure

The reweighting is implemented in

MC_vs_BS_analysis/GBreweighting/fit_GBReweighter.py

1. Merge stopped_recon, zenaz_recon and base weights from the truth table on event_no for each of MC and data.
2. Split each sample at stopped_score > 0.5, producing four populations: MC_{stopped}, MC_{through}, data_{stopped}, data_{through}.
3. For each class, fit a hep_ml.reweight.GBReweighter on the 2D feature space (zenith_pred, azimuth_pred), mapping MC → data. The fit uses original_weight = base_weight_{MC} and target_weight = base_weight_{data}, so the target distribution is the *physically weighted* data distribution, not the raw event count.
4. To avoid overfitting while keeping every event, a 2-fold cross-reweighting scheme is used: MC and data are each split 50/50 (fold A, fold B). Two GBReweighters are fitted, one on MC_A → data_A and one on MC_B → data_B. Each MC event then receives a gb_weight from the reweighter that did *not* see it. Data events get gb_weight ≡ 1.
5. gb_weight is then globally rescaled so that $\sum \text{base_weight}_{\text{MC}} \cdot \text{gb_weight}$ matches $\sum \text{base_weight}_{\text{data}}$ within the class, so that the rate normalisation is carried by the base weights and the GB factor only fixes the shape.
6. The final per-event weight is final_weight = base_weight · gb_weight.

The output is two CSV files (one per class) with columns event_no, source (mc/data), base_weight, gb_weight, final_weight.

GBReweighter hyperparameters. `n_estimators=80`, `learning_rate=0.1`, `max_depth=3`, `min_samples_leaf=200`, `subsample=0.6`.

4 Sanity check: MC vs data classifier ROC

As a closure test, a scikit-learn GradientBoostingClassifier (50 trees, depth 3) is trained to distinguish MC from data on the same 2D feature space (`zenith_pred`, `azimuth_pred`), once using `base_weight` and once using `final_weight`. A 50/50 train/test split is used and both AUC and ROC are computed with `sample_weight`. The result is saved to

`plots/roc_mc_vs_data_before_after.png`

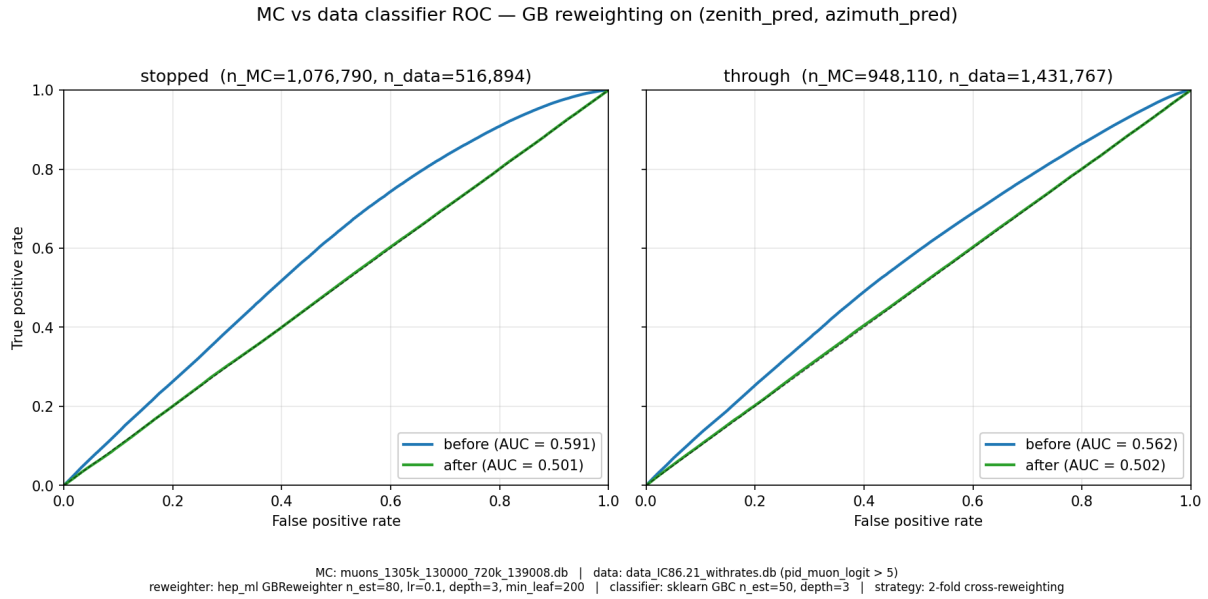


Figure 1: MC vs data classifier ROC, one subplot per class (stopped and through), showing the curve before (blue) and after (green) GB reweighting on (`zenith_pred`, `azimuth_pred`). The dashed diagonal is random ($AUC = 0.5$). Event counts n_{MC} and n_{data} per class are shown in the subplot titles, and the footer lists the data and MC sources as well as the reweighter/classifier hyperparameters.

Interpretation

The *before* AUC is small (around 0.55) for both classes. This is **not** a sign that the sanity-check classifier is weak — it is the physical statement that the 2D marginal of (`zenith_pred`, `azimuth_pred`) is already quite similar between (weighted) MC and (weighted) data. A two-feature BDT essentially has nowhere to go. After the cross-reweighting the AUC drops further towards the random baseline 0.5, confirming that the reweighter closes the small residual gap in the same 2D space it was fitted on.

A direct comparison with a full-pulse DynEdge MC vs BS classifier

`scripts/separate_mc_bs_with_dynedge.py`

would produce a much higher AUC, but that number measures something different: it quantifies how distinguishable MC and data are on the *full pulse image*, i.e. also through all the variables we deliberately did *not* reweight on (energy, position, charge, hit topology, ...). Those variables are the next step in the analysis: they act as blind tests of how faithfully MC reproduces data, now that direction has been reweighted away.

5 Status and next step

Done.

- Stopped/through recon for MC and for PID-filtered data.
- Zenith/azimuth recon for MC and for PID-filtered data, using Inar's direction transformer with a forced position-mode pipeline to be identical on both samples.
- Class split at stopped_score > 0.5.
- 2-fold cross GB reweighting on (zenith_pred, azimuth_pred) per class, with MC base weight norm_class_this_db_osc_weight and data base weight subrun_weight.
- ROC closure check before/after reweighting, one subplot per class.

Next. A plot script that compares MC and data on the test variables (energy_pred, position_z_pred, track length, Q_{tot} , N_{hits} , t_{std} , DOM x, y, z , rde, pmt_area, saturation fraction, HLC/SLC charge) using final_weight, and a second BDT trained on those variables to quantify the remaining MC/data discrepancy outside the reweighted directional subspace.